# Lain ZIP: Checkable and Revisable Memory for Apertus

Hack Apertus 2026 / Track 2B / epsilon-lain / Evidence update: 15 October 2026
AI-assisted implementation and analysis. A research prototype with bounded claims.


## Abstract

A persistent note can preserve an error as easily as a useful discovery. Lain ZIP makes memory revision inspectable: preserve the original response, challenge a saved rule, ask Apertus for a decision and replacement, audit the formal claim, save a version, and compare later use. ZIP denotes a reusable external knowledge record; ordinary ZIP archives are only the delivery format.

A frozen Apertus 70B API pilot repaired inherited errors in three of four strict complete workflows. Content correctness was 12/12 without a note, 10/12 with the old note, and 12/12 with the revision. Separate local Apertus 0.5B training shows limited positive results: four frozen selector batches each favor the trained selector, while a 192-update mathematical writer has a positive periodic evaluation aggregate but only a small final-checkpoint advantage. Student-written summaries have mixed effects. These are distinct experiments, not a pooled benchmark.


## Implemented contribution

The deliverable combines a runnable browser prototype, real model outputs, versioned notes, exact rational polynomial checks, interruption-aware receipts, paired memory conditions, training source, and seven adapter files. It exposes successes and failures without filling in model answers. This is a study of reliable memory operations, rather than a new mathematical principle or a claim of full autonomous learning.

```
Saved note -> fixed challenge -> model-authored checks
-> KEEP / RETRACT -> independent replacement
-> exact audit -> versioned save/readback -> later task
```

## Two model settings, two evidence levels

| Setting | Purpose | Evidence available |
|---|---|---|
| Apertus v1.5 70B API | Memory repair and controller pilot | Untouched raw API archives |
| Apertus v1.1 0.5B local | Writer, selector and memory training | Compact run summaries; source and adapters |

Local findings below are screened historical summaries, not a fresh reproduction of every raw response. Integrity checks on uploaded weights do not independently reproduce their learning benefit.

# An inspectable memory repair

## Actual recorded example: inherited 435 rule

```
Old note: 435 * v * v = 435^2
Fixed witness: v = 0
Model checks: left ZERO / right POSITIVE
Model action: RETRACT
Replacement: 435 * v^2
Stored form: 435 * v**2
```

The witness was chosen by the host before the experiment. Separate model calls precede the withdrawal decision. The model writes the replacement; the host checks its polynomial coefficients. Display normalization records the conversion of superscript notation without changing coefficients. Original text remains preserved.

## Later task: simplify 2 * 435 * z * z

| Memory condition | Actual answer | Content correct |
|---|---|---|
| Old note | 2 * 435^2 * z^2 | No |
| Revised note | 870 * z**2 | Yes |
| No note | 870 * z**2 | Yes |

This removes bad-memory interference. It does not improve content correctness over the already-correct no-note baseline.

## Frozen pilot denominators

| Measurement | Result |
|---|---|
| Coverage | 5 rule units / 6 episodes |
| Wrong-note withdrawals and valid replacements | 4/4 wrong-note episodes |
| Strict two-follow-up complete repair workflows | 3/4 |
| False withdrawal of correct-note controls | 0/2 |
| Later content correct: no / old / revised note | 12/12 / 10/12 / 12/12 |
| Strict output success: no / old / revised note | 10/12 / 10/12 / 12/12 |

The two repaired errors repeat one underlying 435 task. One mathematically valid episode fails the fixed sign-token interface. A separate 15 October live smoke test repairs 435 but wrongly retracts a correct 315 note without a grounded conflict; it is not added to the frozen pilot score. Formal expression acceptance does not certify reflective prose.

# What local parameter training changed

## Mathematical writer: 192 committed updates

Run 362b296b uses a frozen 438,349,352-parameter Apertus base with rank-8 LoRA on query/value projections in the final four attention layers. The adapter has 106,496 trainable parameters. Other manager, route and summary states stay fixed. The course records 630 actual student generations and no held-out optimizer updates.

The policy loss uses actual sampled student token log probabilities, a reward baseline of 0.5, and a fixed 96-token denominator. AdamW uses learning rate 0.0001, zero weight decay and gradient clipping at 1. A 0.25-weight rehearsal term replays verified student training answers. This is not a pure reward-only course: the student-answer replay term is part of the method.

| Writer evaluation | Trained | Paired initial | Difference |
|---|---|---|---|
| 24 periodic objects: managed routes | 44.41% | 25.02% | +19.39 pp |
| 24 periodic objects: no summary | 43.66% | 24.26% | +19.40 pp |
| Final checkpoint, 4 fresh objects: managed | 54.24% | 53.58% | +0.67 pp |
| Final checkpoint, 4 fresh objects: no summary | 57.86% | 56.82% | +1.04 pp |

The metric is probability mass assigned to actually verified successful method routes. It is not observed answer accuracy. The 24 periodic objects span different milestones; they are not a fixed-test learning curve or the final checkpoint result. The final four-object contrast is small.

## Longer paired reward training

| Course | Training episodes per arm | Verified: reward on / off | On-arm updates |
|---|---|---|---|
| Scale 309e8004 | 512 | 346 / 160 | 377 |
| Warm-up b1e08b63 | 128 | 70 / 47 | 115 |
| Replication d27d9608 | 128 | 54 / 43 | 101 |

These are TRAIN event counts, not held-out accuracy. The zero-reward control retains its initial writer. No new standalone mathematical principles were admitted. Some earlier project phases used reference-answer supervision and teacher review; no project-wide teacher-free or no-SFT claim is made.

# Selectors improve; written summaries remain mixed

## Four frozen selector batches

Each batch reports eight new objects. The frozen mathematical writer and actual audited route candidates are shared across selector conditions. The host schedules the three routes and verifies the results; the learned selector assigns probability mass. This does not establish autonomous retrieval or independent training replications.

| Run suffix | Trained mass | Initial mass | Difference |
|---|---|---|---|
| 2475c06b | 43.07% | 29.17% | +13.90 pp |
| 709ff875 | 36.42% | 20.83% | +15.58 pp |
| 3f226854 | 48.92% | 41.67% | +7.25 pp |
| 63b01086 | 50.39% | 45.83% | +4.55 pp |
| Descriptive equal-weight mean | 44.70% | 34.38% | +10.32 pp |

There are 32 reported objects; cross-batch uniqueness and all checkpoint bindings were not independently re-audited from raw outputs. Across reported pairs, 16 favor the trained selector, 12 favor the initial selector and 4 tie.

## Six consecutive summary courses: same-head context ablation

| Round / run | Held-out objects | With minus without summary |
|---|---|---|
| 11 / 19c328cb | 4 | 0.00 pp |
| 12 / 464f3215 | 4 | -3.78 pp |
| 13 / 1e35f1a6 | 4 | -2.10 pp |
| 14 / f994839d | 4 | +3.74 pp |
| 15 / 76c8254e | 4 | -1.92 pp |
| 16 / 58af84e4 | 4 | +8.59 pp |
| Descriptive mean across all six | 24 reported | +0.75 pp |

The best round is a useful existence signal, but the full sequence is mixed. These are continuing curricula, not six independent seeds. The program supplies structural scope; autonomous scope discovery is untested. The context ablation includes the adapted reader. Summary prose is not certified as wholly true.

Series aggregates and their component rounds must not be counted twice. Missing nested metrics in compact exports remain unavailable; they are not treated as zero.

# Failure modes define the next experiment

## An outcome-trained controller can learn the wrong policy

The separate 70B API experiment trains a four-action external linear controller for 128 real NumPy REINFORCE/Adam updates; it does not update the API model weights. Six training contexts all favor withdrawal under the realized downstream reward. On four held-out contexts / eight questions, the policy always selects RETRACT, abstains 8/8 times and answers 0/8 correctly.

Learned utility is -0.12, better than uniform action utility -0.56975 but equal to a constant-withdrawal baseline. That comparison exposes a deficient objective and training distribution. It does not show useful memory management. Eight host-authored features include consistency; their contribution is not autonomous discovery.

## Additional local negative controls

| Experiment | Observed result | Interpretation |
|---|---|---|
| One-step self-check | 8/8 clean drafts -> 6/8 clean revisions | Self-check can lose coverage |
| Reflection transfer | 4/8 first / ordinary retry -> 1/8 reflection | Reflection can harm |
| Independent verifier: 128 updates | Initial and trained: 12/24 answer passes | No answer-level gain here |
| Native memory management | 0 verified training actions | Fallback is not learned suspension |
| Conditional v24 editor | 0 eligible episodes / 0 actual editor updates | Saved editor equals initialization |

## Relation to prior work

Reflexion [1] uses verbal reflections as episodic memory without model weight updates. Memory-R1 [2] trains memory management and answering policies with reinforcement learning. SCoRe [3] studies training for self-correction. Lain ZIP is a small implementation and evaluation study at their intersection: inspectable memory revisions, restricted algebra checks, and explicit controls. It does not claim a new general learning algorithm.

## Next decisive test

Balance training cases where retention, revision and withdrawal are each useful; preregister a fresh held-out set; separate final-checkpoint evaluation from periodic probes; measure correct answers and valid derivations alongside route mass; and compare learned management with always-use, always-suspend and matched-budget retry controls. Repeat independent training seeds before a robustness claim.

# Reproduce, inspect and deploy within scope

## Runnable submission and model release

```
make run # Docker; open localhost:7860
make test audit # offline checks and archive audit
make replay # actual recorded 435 example
```

Archive mode works without a key. New inference accepts server-side LLM_NAME, LLM_BASE_URL and LLM_API_KEY for an Apertus endpoint; credentials are not exposed to the browser. Each request gets a pending receipt, raw output is stored before auditing, and interrupted calls are not silently retried. New live runs remain separate from frozen results.

The companion release contains seven adapter files, model-loading utilities, checkpoint metadata, original v15/v24 training modules and a standard-library integrity checker. The conditional editor is explicitly labeled as an initial, untrained artifact. Base model weights and tokenizer are not bundled; the compatible Apertus snapshot must be supplied separately. This submission environment checked adapter bytes/tensors and source bindings, not fresh base-model inference.

Public repository branch:
github.com/epsilon-lain/lain-brain / codex/hack-apertus-model-training-20261015
Runnable entry: track_2b/README-project.md. Model entry: experiments/zip/hack-apertus-2026/. Full historical screening: its evidence directory.

## Evidence and sovereignty boundary

Three unchanged API archives preserve 245 responses in 247 attempts: 72 + 63 + 110 responses, with two uncertain/interrupted attempts. The 15 October prototype adds 14 separate smoke-test responses. Local-history screening covers 113 run directories represented by compact result records; duplicate receipts and series summaries are not independent replications.

The Python prototype and local exact auditor can run on premises. The recorded API pilot used CSCS inference [4]. An air-gapped installation requires locally supplied Apertus weights and an inference endpoint; the hosted API pilot itself is not an air-gapped deployment. No complete Swiss storage, energy-efficiency or production-scale claim is established.

## References

[1] Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. 2023. arxiv.org/abs/2303.11366.
[2] Yan et al. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. 2025; revised 2026. arxiv.org/abs/2508.19828.
[3] Kumar et al. Training Language Models to Self-Correct via Reinforcement Learning (SCoRe). 2024. arxiv.org/abs/2409.12917.
[4] CSCS inference API documentation: docs.cscs.ch/services/inference/api/.

Evidence cut-off: 15 October 2026. Restricted real-polynomial equalities and small samples. No statistical significance, broad transfer, new mathematical principle, or full autonomous learning claim.